

LINFO2132 - Phase 3: Semantic Analysis Report

1. Current Progress The semantic analysis phase has been successfully implemented. The compiler now performs a complete recursive-descent traversal of the Abstract Syntax Tree (AST) generated by the parser. It successfully detects and halts execution (with a non-zero exit code) for all 7 required semantic errors : `TypeError`, `CollectionError`, `OperatorError`, `ArgumentError`, `MissingConditionError`, `ReturnError`, and `ScopeError` .

2. Implementation Choices

- **Scope Management:** I implemented a `Stack<Map<String, String>>` to handle lexical scoping. Every time the analyzer enters a new `BlockNode` or `FunctionNode`, a new `HashMap` is pushed onto the stack. This allows for efficient $O(1)$ variable lookup in the current scope and straightforward shadowing of global variables.
- **Function and Collection Registration:** I used two separate `HashMap` structures (`functionSignatures` and `collectionTypes`) to store global definitions. This choice ensures fast verification of argument counts/types during function calls (`ArgumentError`) and prevents duplicate collection definitions (`CollectionError`).

3. Improvements to Previous Phases During the integration of the Semantic Analyzer, I had to refactor and improve several components from Phase 2:

- **Parser Flexibility:** The `parseFactor()` method in `Parser.java` was refactored to correctly identify `true` and `false` string values and convert them directly into `BooleanNodes`, bypassing previous strict token-type limitations that caused `Syntax Error: Unexpected symbol 'true'`.
- **AST Node Accessibility:** I updated the visibility modifiers of fields in all AST Node classes to `public` to allow the `SemanticAnalyzer` to access their properties across different packages.
- **Print Methods:** I implemented the missing `@Override public void print(String)` methods for all newly added AST nodes (`BooleanNode`, `CollectionNode`, `FunctionNode`, etc.) to comply with the `ASTNode` abstract class requirements from Phase 2.

4. Updated Grammar Rules The parser was expanded to support the new syntactic elements introduced in this phase:

- Functions: `def [TYPE|void] IDENTIFIER ( [PARAMETERS] ) { [BLOCK] }`
- Collections: `coll IDENTIFIER { [DECLARATIONS] }`
- Returns: `return [EXPRESSION];`
- Function Calls: `IDENTIFIER ( [ARGUMENTS] )`

